In-Ear System Architecture Documentation — DRAFT

Version 0.1 — February 27, 2026

Document Control

Document ID	Pending
Version	0.1
Status	Draft
Prepared by	Polymer Bionics Limited
Approved by	
Approval Date	
Review Date	

Revision History

Version	Date	Author	Summary
0.1	February 27, 2026	Polymer Bionics Limited	Initial draft issued for internal review and QMS approval.

Contents

1 System Overview

The BioBoard is a modular, **platform-agnostic** biosignal acquisition platform built around the **Teensy 4.1** and **TI ADS1299** 24-bit EEG front-end. A **standalone Qt Connector App** bridges hardware to **Lab Streaming Layer (LSL)**, enabling any LSL-compatible software (OpenBCI GUI, MATLAB, LabRecorder, etc.) to consume the data.

Four architectural layers:

L1 Hardware — Teensy 4.1 + ADS1299 + sensors

L2 Firmware — Compact Protocol v1.0 over USB serial

L3 Connector — Qt Connector App (USB → LSL)

L4 Consumers — Open Ephys and any other LSL-compatible software

1.1 End-to-End Data Flow

Primary production path (recommended):

- **Connector-first path:** Qt Connector App reads serial data and publishes LSL streams directly. Open Ephys can then subscribe as an LSL consumer (same as MATLAB, Python, LabRecorder, etc.).
- **Single-parser architecture:** No direct firmware → Open Ephys serial path in the primary design. This avoids maintaining two separate protocol parsers (Connector + Open Ephys).
- **Event round-trip (optional):** Open Ephys can publish markers/events back into LSL (e.g., Paradigm Bridge), while external tools such as PsychoPy can also publish behavioral/event streams to LSL.

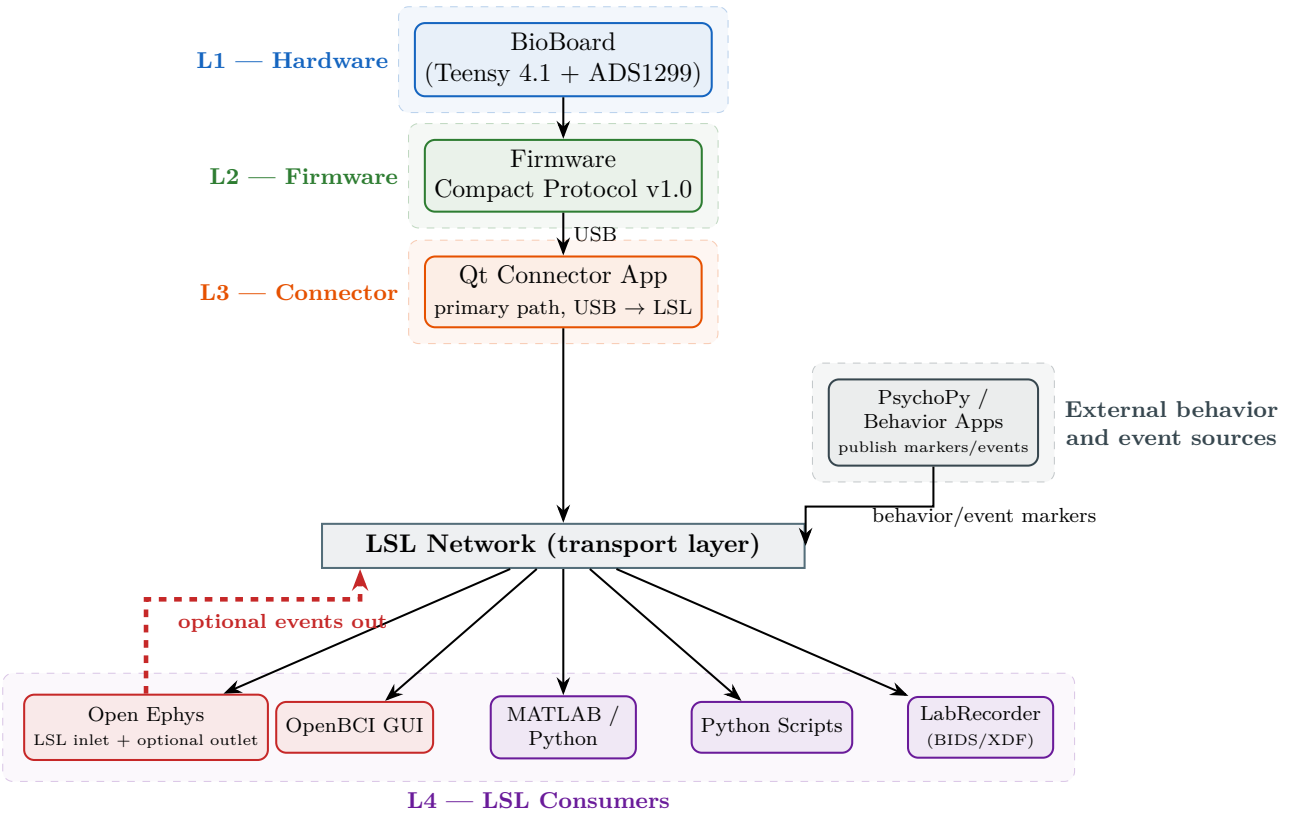

Figure 1: Primary architecture: Qt Connector App is the single serial parser and publishes LSL streams consumed by Open Ephys and other tools. External behavior/event tools (e.g., PsychoPy) can publish markers to LSL, and Open Ephys can optionally publish events back to LSL (dashed). The LSL network is transport infrastructure, not an L4 consumer.

2 Layer 1 — Hardware (BioBoard)

2.1 Core Components

Table 1: BioBoard hardware components

Component	Manufacturer / Part	Function
MCU	Teensy 4.1 (ARM Cortex-M7, 600 MHz) Texas Instruments	Main controller; USB 2.0 Full Speed
EEG ADC	ADS1299 datasheet Analog Devices, Inc.	Biosignal front-end; SPI interface; configurable gain 1×–24×
Accelerometer	ADXL367 datasheet Analog Devices, Inc.	Motion/orientation; low-noise MEMS
PPG / SpO_2	MAXM86161 datasheet Texas Instruments	Integrated PPG + SpO_2; single optical module
Temperature	TMP117 datasheet	High-precision digital temperature sensor
Connection	USB 2.0	Serial at 2 Mbaud (2 000 000 bps)

2.2 LSL Stream Metadata Reference

The Qt Connector Application publishes two LSL streams per device:

Table 2: LSL StreamInfo constructor parameters

Property	EEG Stream	Marker Stream	Notes
name	`"InEarEEG"`	`"InEarEEG_Markers"`	Unique per device on network
type	`"EEG"`	`"Markers"`	Enables LabRecorder / BIDS auto-detect
channel_count	8	1	8 EEG channels; 1 event channel
nominal_srate	1000.0	0	0 = `IRREGULAR_RATE`
channel_format	`cf_float32`	`cf_string`	μV for EEG; labels for markers
source_id	`"inear_001"`	`"inear_001_markers"`	Stable across sessions

3 Layer 3 — Connector Applications

The **`LSL Connector Application`** is the **primary production connector** — a standalone Qt desktop app that bridges Teensy hardware to LSL, completely independent of Open Ephys or any other recording software.

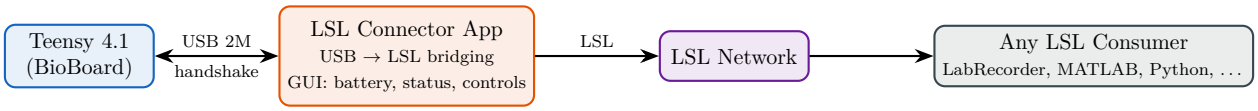

Figure 2: LSL Connector Application data flow with two-way USB handshake.

3.1 Minimum Viable Product (MVP)

User Interface

1. **Scan Button** — Searches for connected Teensy devices (vendor/product ID filtering).
2. **Stream Controls** — "Open LSL" / "Close LSL" toggle.
3. **Status Display** — Battery level, connection state, stream health.

Core Connectivity & Data Flow

1. **USB Scanning** — Enumerates ports, filters by Teensy IDs.
2. **Protocol Parsing** — Demultiplexes incoming serial stream per the Compact Protocol.
3. **LSL Bridging** — Pushes parsed samples into an LSL outlet with correct metadata.

Two-Way Communication & Handshake Protocol A structured 4-step handshake precedes data streaming:

Table 3: Handshake protocol steps

Step	Phase	Description
1	Device Recognition	Ping/response to confirm Teensy hardware identity.
2	Firmware Recognition	Version query; mismatched versions trigger a warning.
3	Pairing Handshake	Establishes exclusive session with the specific Teensy.
4	Information Exchange	Dynamic retrieval of device configuration: • Required number of LSL channels • Battery status and charge level • Sensor metadata (which sensors are present) • Current sample rate and gain settings

Table 4: Error handling requirements

Condition	Required Behaviour
USB disconnect / timeout	Detect within 500 ms; auto-pause LSL; attempt reconnect with exponential back-off.
Malformed packets	Discard (checksum failure); increment error counter; continue streaming valid data.
Buffer overruns	Drop oldest samples when ring buffer exceeds capacity; log warning with drop count.
Hardware failure flags	Parse Teensy error codes; display with severity colour coding; optionally pause on critical errors.

Error & Special Situation Handling

3.2 Future Roadmap

Expanded Hardware Support

- **SD Card Access** — Read/manage SD card files via Teensy USB.
- **Bluetooth** — Wireless streaming via RFCOMM/SPP transport.

UI Enhancements

- **Connection Toggle** — Seamless USB/Bluetooth switching.
- **Multi-device support** — Simultaneous connections, each with its own LSL outlet.
- **Config persistence** — Save/load device profiles.

4 Layer 4 — Consumer Software & Open Ephys Plugins

The LSL streams from the Qt Connector App can be consumed by any LSL-compatible software (OpenBCI GUI, MATLAB, LabRecorder, Python scripts, or any `liblsl` binding). For **Open Ephys** users, in-house C++ plugins provide enhanced integration on top of the LSL-first workflow.

4.1 Open Ephys Plugin Inventory (Optional Integrations)

For users who prefer extended Open Ephys workflows, the following plugins are available. The primary production path remains Qt Connector App → LSL → Open Ephys (LSL inlet).

Table 5: Complete plugin inventory (optional Open Ephys integrations)

Plugin	Type	Function
In-house plugins (written by us)		
InEar Teensy Source	DataThread	Direct USB; Compact Protocol v1.0 (8-ch, 32–38 B)
InEar Teensy Optimised	DataThread	Direct USB; legacy variable 26–55 B protocol
Custom IC Source	DataThread	Generic serial IC reader (any protocol)
OpenBCI Cyton	DataThread	Cyton/Daisy 8/16-ch wireless EEG
EDF File Source	FileSource	EDF/BDF/EDF+/CSV file playback
LSL Outlet	Processor	Standalone LSL streaming output
Paradigm Bridge	Processor	TCP experiment control + TTL injection
Open Ephys community plugins (used as dependencies)		
Lab Streaming Layer I/O	Mixed	LSL inlet (source) + outlet (sink)
Record Node	Sink	Binary / NWB / Open Ephys format recording
LFP Viewer	Visualiser	Real-time waveform display
File Reader	Source	Built-in file playback (extended by EDF plugin)

4.2 Source Plugins — Serial Data Reader (Legacy / Development)

The InEar Teensy Source plugin is a **custom in-house plugin** that reads serial data directly from the BioBoard over USB. It is kept for legacy compatibility and low-level debugging, but is not part of the primary production architecture (which uses the Qt Connector as the single parser).

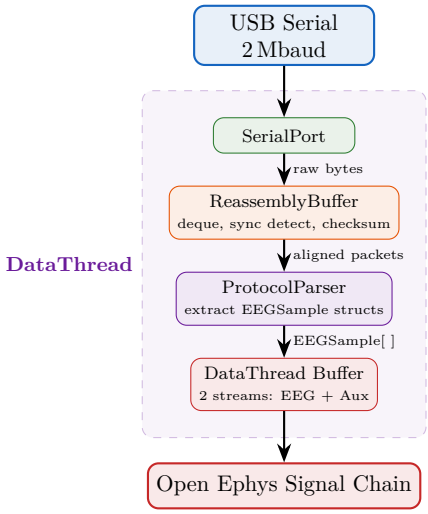

Figure 3: Internal architecture of the InEar Teensy source plugin.

4.2.1 Packet Reassembly

USB transfers arrive in arbitrary-sized chunks. The `ReassemblyBuffer` uses a `std::deque<uint8_t>` to:

1. Scan for sync word `0xA5 5A`.
2. Read expected length from TYPE+TTL byte.
3. Validate XOR checksum.
4. Extract validated packet; discard pre-sync bytes.
5. Detect lost packets via timestamp gaps.

4.2.2 Output Data Streams

Each source plugin registers **two data streams** with Open Ephys:

Stream	Channels	Rate	Contents
EEG	8	1000 Hz	Channels 1–8, scaled to μV
Aux	varies	1000 Hz	Accel, PPG+SpO_2, Temp, Battery (sample-and-hold)

4.2.3 Simulation Mode

Both source plugins include simulation mode (toggled in editor UI):

- **EEG:** Sine waves at 3, 7, 11, 15, 19 Hz (one per channel)
- **PPG:** 72 BPM synthetic waveform with Red/IR/Green variations
- **Accel:** Slow sinusoidal motion
- **Sync:** Xorshift32 PRNG-driven digital pulse

4.3 OpenBCI Cyton Plugin

The system also supports the **OpenBCI Cyton** board (8-channel wireless EEG):

Table 6: OpenBCI Cyton vs. BioBoard comparison

Feature	BioBoard (InEar)	OpenBCI Cyton
ADC	ADS1299 (same chip)	ADS1299 (same chip)
Channels	8 EEG + aux (Accel, PPG, Temp, Bat)	8 EEG (+8 Daisy) + 3 accel
Sample rate	1000 Hz	250 Hz
Connection	USB wired, 2 Mbaud	2.4 GHz radio, 115 200 baud
Packet size	32–38 B (Compact Protocol)	33 B
Extra sensors	PPG+SpO_2, Temp, Battery	Accelerometer only

4.4 Paradigm Bridge — Experiment Control

The `Paradigm Bridge` is a **filter processor** providing bidirectional TCP control between experiment software (PsychoPy, MATLAB) and Open Ephys.

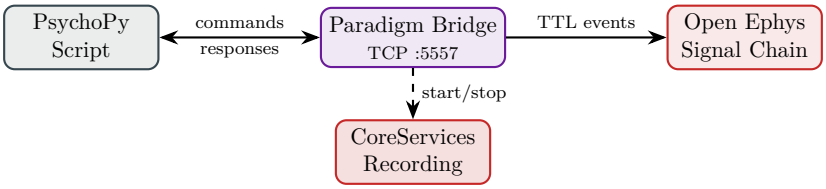

Figure 4: Paradigm Bridge bidirectional control flow.

4.5 LSL Integration

- **LSL Inlet** (DataThread) — Receives any LSL stream as an Open Ephys source.
- **LSL Outlet** (Processor) — Streams data and markers from Open Ephys to external consumers.

4.6 EDF File Source

The `EDF File Source` extends the built-in File Reader to support EDF, BDF, EDF+, CSV, and XZ-compressed CSV files for offline replay.

5 Signal Chain Configurations

5.1 Primary Path — Qt Connector App (Standalone)

For users with or without Open Ephys:

```
[BioBoard] -> USB -> [Qt Connector App] -> LSL network
                                              |
              +-------------+----------------+---------------------+
              |             |                |                     |
      [Open Ephys Inlet] [OpenBCI GUI] [LabRecorder]        [MATLAB/Python]
      (visualize/record)               (XDF file)

[PsychoPy / behavior scripts] -> LSL network (markers/events)
```

5.2 Open Ephys Path — LSL Inlet + Optional LSL Outlet

Open Ephys subscribes to the Connector App stream via LSL (no direct serial parser in Open Ephys on the primary architecture), visualises/records internally, and can optionally publish events/markers back to LSL:

```
[BioBoard] -> USB -> [Qt Connector App] -> LSL network
                                              |
                                      [Open Ephys Inlet]
                                              |
              +-------------------------+-------------------------+
              |                         |                         |
        [LFP Viewer]   [Paradigm Bridge] -> [Record Node]   [LSL Outlet]
                                                                  |
                                                              LSL network
                                                                  |
                                                [LabRecorder] [MATLAB] [Python]
```

5.3 Offline Replay with Annotations

Replay a previously recorded EDF file with post-hoc annotation:

```
[File Reader + EDF Source] -> [Paradigm Bridge] -> [LFP Viewer]
```

5.4 Multi-Device with LSL

Multiple BioBoards and/or third-party devices, each with their own Qt Connector App instance:

```
[BioBoard A] -> [Qt Connector App 1] -> LSL "InEarEEG_A"
[BioBoard B] -> [Qt Connector App 2] -> LSL "InEarEEG_B"   -> [LabRecorder]
[OpenBCI Cyton] -> [OpenBCI GUI] -----> LSL "OpenBCI"      -> [LabRecorder]
```

6 Build System

All C++ plugins use the standard Open Ephys CMake template:

```
cd OEPlugins/<plugin-name>/Build
cmake -G "Visual Studio 17 2022" -A x64 ..
cmake --build . --config Release
cmake --install . --config Release
# DLL installed to: plugin-GUI/Build/Release/plugins/
```

Listing 1: Building a plugin

Table 7: Build requirements

Tool	**Version**
CMake	≥ 3.15
Visual Studio	2022 (MSVC 17)
Open Ephys Plugin API	v10
Open Ephys GUI	v1.0.1 (plugin-GUI submodule)